

PRÁCTICA 4

Expresiones regulares en C++

Factor de ponderación: 9

Semana del 5 al 9 de octubre 2026

1. Objetivos

El objetivo de la práctica es continuar trabajando con expresiones regulares, aunque en este caso en lugar de usar herramientas de simulación como las utilizadas en la práctica anterior, utilizaremos expresiones regulares dentro de código en C++. En el ámbito de la teoría de autómatas y los lenguajes formales, una expresión regular (*regular expression* o *regex*) es un mecanismo formal que permite describir lenguajes regulares. Es decir, toda expresión regular representa a un lenguaje regular. Desde un punto de vista más general podríamos decir que una expresión regular es una secuencia de caracteres que conforma un patrón de búsqueda. Las expresiones regulares proporcionan una manera muy flexible de buscar o reconocer cadenas en un texto.

Los principales lenguajes de programación tienen soporte para la utilización de expresiones regulares (C++, Python, Java, Ruby, etc.). Esto se debe a que habitualmente los programadores necesitan trabajar con cadenas de texto (strings) para validar los datos ingresados por el usuario, validar formatos de una URL, de un correo electrónico, reemplazar palabras en párrafos, etc.

A modo de resumen, podríamos decir que los usos principales de las expresiones regulares son los siguientes [2]:

- **Buscar** elementos particulares (determinadas subcadenas) dentro de un texto principal (cadena). Por ejemplo, es posible identificar todas las direcciones de correo electrónico presentes en un archivo.
- **Reemplazar** elementos particulares (determinadas subcadenas) dentro de un texto

principal (cadena). Por ejemplo, es posible querer eliminar todos los comentarios de un fichero de código fuente.

- **Validar** entradas comprobando que una determinada cadena cumple con un patrón específico. Por ejemplo, puedes querer verificar que una contraseña cumpla con ciertos criterios, tales como: una combinación de mayúsculas y minúsculas, dígitos, signos de puntuación, etc.
- **Seleccionar** elementos en base a ciertos patrones. Por ejemplo, es posible si quieres procesar ciertos archivos en un directorio, pero solo si cumplen condiciones particulares.
- **Formatear** contenido en un archivo. Por ejemplo, puedes querer mejorar el estilo de tu código asegurándote de que entre cualquier operador binario hay espacio en blanco tanto a la izquierda como a la derecha.

Tal y como ya hemos mencionado, la librería estándar de C++ proporciona soporte para el uso de expresiones regulares a través del módulo `<regex>`. El objetivo de esta práctica será precisamente aprender a utilizar expresiones regulares en C++ a través de las clases y las funcionalidades proporcionadas en `<regex>`. Al igual que en las prácticas anteriores, también se propone que el alumnado utilice este ejercicio para poner en práctica aspectos generales relacionados con el desarrollo de programas en C++. Algunos de estos principios que enfatizaremos en esta práctica serán los siguientes:

- **Programación orientada a objetos:** es fundamental identificar y definir clases y objetos que permitan modelar adecuadamente el escenario de trabajo que se plantea.
- **Diseño modular:** el programa debiera escribirse de modo que las diferentes funcionalidades que se precisen sean encapsuladas en métodos concretos cuya extensión textual se mantenga acotada.
- **Pautas de estilo:** es imprescindible ceñirse al formato para la escritura de programas en C++ que se propone en esta asignatura [11]. Algunos de los principales criterios de estilo ya se han descrito reiteradamente en las prácticas anteriores de esta asignatura.
- **Expresiones regulares:** se utilizará la librería `<regex>` para hacer uso de expresiones regulares en programas escritos en C++.
- Compilación de programas utilizando `make` [9, 10].

2. Regex

La librería estándar de C++ suministra soporte para expresiones regulares a través de la librería de expresiones regulares (desde C++11). Para utilizar esa librería es necesario incluir en los programas C++ el fichero de cabecera `<regex>`.

Para comenzar a utilizar expresiones regulares en C++ es importante conocer las operaciones básicas proporcionadas por esta librería:

- **regex_match**: devuelve si la secuencia de caracteres completa coincidió con la expresión regular especificada, opcionalmente capturando en un objeto coincidente.
- **regex_search**: devuelve si una parte de la secuencia de caracteres coincidió con la expresión regular, opcionalmente capturando en un objeto coincidente.
- **regex_replace**: devuelve la secuencia de caracteres de entrada modificada por una expresión regular a través de una cadena de formato de reemplazo.

Tanto **regex_match** como **regex_search** son funciones coincidentes. La principal diferencia es que **regex_match** comprueba si una expresión regular coincide con **toda** la cadena de destino mientras que **regex_search** se detiene cuando encuentra la primera coincidencia y no realiza múltiples búsquedas. Por su parte, **regex_replace** realiza múltiples búsquedas, reemplazando la cadena coincidente encontrada con la cadena de reemplazamiento especificada.

A la hora de llevar a cabo estas operaciones de búsqueda y/o reemplazamiento, intervienen los elementos siguientes:

- *Target sequence*: secuencia destino o *string* sobre el cual se realizará la búsqueda del patrón.
- *Regular expression*: expresión regular que define el patrón a encontrar en la secuencia destino.
- *Matched array*: estructura en la que se almacena la información sobre el *matching* o emparejamiento realizado.
- *Replacement string*: cadena de reemplazo que sustituirá al patrón especificado dentro de la secuencia destino.

Para poder especificar el patrón de búsqueda se utiliza una sintaxis de expresiones regulares que, además de incluir los operadores básicos de unión, concatenación y asterisco, admite extensiones de dichos operadores.

Algunos de estos operadores ya se introdujeron en clase o se utilizaron en la práctica anterior, pero existen algunos otros que además de ser válidos en la librería **regex** de C++ también lo son en algunas de las librerías disponibles en otros lenguajes de programación.

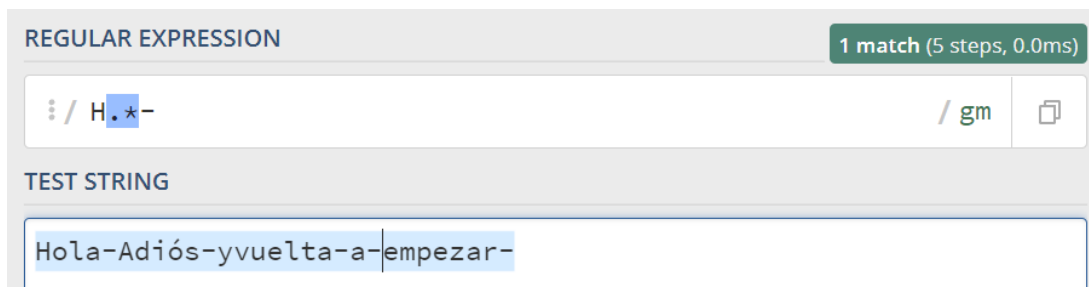
A continuación se relacionan algunos de los principales operadores extendidos que más se suelen utilizar para definir expresiones regulares:

<code>[A – Z]</code>	Cualquier letra mayúscula
<code>[0 – 9]</code>	Cualquier dígito
<code>[A – Za – za0 – 9]</code>	Cualquier letra o dígito
<code>.</code>	Cualquier caracter (excepto el salto de línea)
<code>+</code>	Una o más ocurrencias
<code>*</code>	Cero o más ocurrencias
<code>?</code>	Cero o una única ocurrencia
<code>{m}</code>	Exactamente <i>m</i> ocurrencias
<code>{m,n}</code>	Entre <i>m</i> y <i>n</i> ocurrencias (ambas incluidas)
<code> </code>	Una cosa u otra (or)
<code>^</code>	Primera ocurrencia (comienzo del patrón)
<code>\$</code>	Última ocurrencia (final del patrón)
<code>\+, *, \\\</code>	Los caracteres especiales hay que antecederlos de \
<code>\d</code>	Cualquier dígito
<code>\D</code>	Cualquier caracter que no sea un dígito
<code>\w</code>	Cualquier letra o dígito
<code>\W</code>	Cualquier caracter que no sea letra ni dígito
<code>\s</code>	Cualquier blanco
<code>\S</code>	Cualquier caracter que no sea un blanco
<code>()</code>	Captura de grupos y/o subgrupos

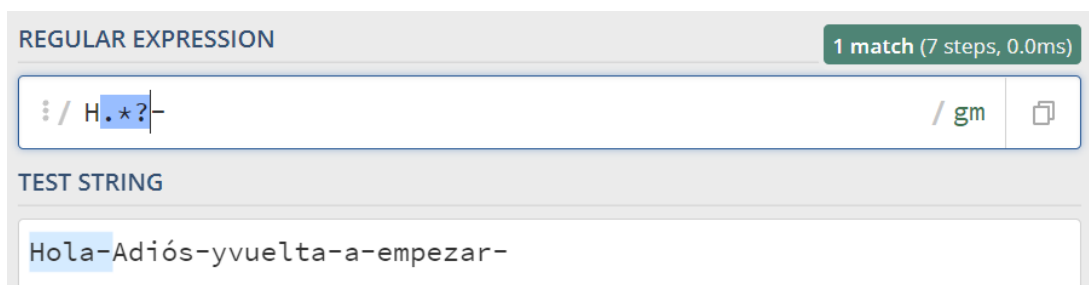
Para entender mejor y poner en práctica la sintaxis de las expresiones regulares asegúrese de que ya ha estudiado el tutorial interactivo disponible en [4]. También se recomienda la lectura de los materiales disponibles en [2, 5].

En lo que se refiere a la sintaxis, es importante hacer inciso en dos aspectos: el comportamiento “*greedy*” del operador `*` y la utilización de los agrupamientos para luego acceder a partes concretas de la secuencia coincidente.

En primer lugar, es importante tener en cuenta que los cuantificadores de repetición por defecto tienen un comportamiento ávido (*greedy*). Para mostrar este comportamiento se ha utilizado una herramienta para el testeo online con expresiones regulares [8]. Si nos fijamos en la expresión regular siguiente (cadenas que empiezan por H seguida de cero o más caracteres y terminan en guión), veremos que, por defecto, el `.*` casará con toda la cadena, llegando hasta el último guión (-). A este comportamiento se le llama ávido o *greedy* porque casa con la mayor cadena posible [6].



Sin embargo, si utilizáramos `. * ?` estaríamos usando un cuantificador perezoso o *lazy*. De esta forma, al realizar el emparejamiento se pararía con la primera subcadena (subcadena más corta) que cumpla con el patrón especificado [7].



Por otro lado, la utilización de grupos sirve para capturar partes de la subcadena que ha casado con el patrón de búsqueda. Tomando como ejemplo el [código siguiente](#):

```

1 #include <iostream>
2 #include <regex>
3 #include <string>
4
5 int main() {
6     // Target sequence
7     std::string target_s = "I am looking for GeeksForGeeks articles";
8
9     // An object of regex for pattern to be searched
10    std::regex expression("Geeks[a-zA-Z]+");
11
12    // smatch for storing the matches
13    std::smatch matches;
14
15    // regex_search() for searching the regex pattern
16    // 'expression' in the string 'target_s'.
17    regex_search(target_s, matches, expression);
18
19    for (auto match : matches)
20        std::cout << match << std::endl;
21
22    return 0;
23 }
```

En el array `matches` se almacena la información de las coincidencias (*matchings*) obtenidas. En este caso, como no se han usado agrupamientos, la única cadena mostrada en el bucle sería:

```
1 GeeksForGeeks
```

Si, por el contrario, en la expresión regular se hubiera usado un agrupamiento para “almacenar” la cadena que va entre *Geeks* y el primer caracter diferente de una letra:

```
1 regex r("Geeks ([a-zA-Z]+) ")
```

Entonces el bucle habría mostrado dos cadenas, la cadena completa del emparejamiento y aquella parte que ha casado con el agrupamiento especificado:

```
1 GeeksForGeeks
2 ForGeeks
```

3. Ejercicio práctico

Para poner en práctica lo estudiado hasta el momento, se desarrollará un programa en C++ que analice el contenido de ficheros de código HTML haciendo uso de expresiones regulares. El programa recibirá por línea de comandos el nombre del fichero de entrada y el nombre del fichero de salida:

```
1 ./p04_html_analyzer pagina.html esquema.txt
```

En caso de ejecutarse sin parámetros, el programa mostrará un mensaje de error indicando el modo correcto de ejecución, indicando los parámetros necesarios y el significado de cada uno.

El fichero de entrada será un fichero de código HTML sintácticamente correcto. El fichero de salida resumirá la estructura del fichero de entrada: etiquetas indicadas, atributos, estructura básica del documento y comentarios. El fichero de salida no se creará directamente mientras se analiza el fichero de entrada sino que **se creará al menos una clase que represente y almacene la estructura general del código HTML analizado.**

Se deberá hacer uso de expresiones regulares para analizar el código fuente de entrada y extraer la información siguiente:

- **Etiquetas:** Se deberán detectar tanto las etiquetas de apertura como las etiquetas de cierre, cuando estas existan. Las etiquetas que no requieren cierre, como `<img>`, se considerarán únicamente como etiquetas de apertura.

Las etiquetas que se deberán procesar son: `<html>`, `<head>`, `<title>`, `<body>`, `<h1>`, `<p>`, `<a>` y `<img>`. No será necesario detectar otras etiquetas HTML diferentes de las indicadas anteriormente.

Para cada etiqueta se almacenará su nombre y la línea en la que aparece. En el caso de las etiquetas de apertura, se almacenará además si tiene atributos y el número de atributos. Por ejemplo:

```
<a href="https://ull.es" class="enlace">ULL</a>
```

La salida esperada en el análisis sería:

```
[Line 12] TAG: a
          ATTRIBUTES: 2
```

- **Atributos:** Se deben detectar todos los atributos en la forma `nombre="valor"`. Por ejemplo:

```

```

La salida esperada en el análisis sería:

```
TAG: img
ATTRIBUTES:
  src = "foto.jpg"
  alt = "Foto de la universidad"
  width = "300"
```

- **Estructura básica del documento:** Detectar si existen las etiquetas `<html>`, `<head>` y `<body>`, así como `<!DOCTYPE html>`
- **Comentarios:** Detectar todos los comentarios, ya sean en una línea o multilínea:

```
1 <!-- Este es un comentario -->
2
3 <!--
4 Este es un comentario
5 multilinea
6 -->
```

Se almacenarán la línea o líneas que ocupa cada comentario, así como el propio comentario. Además, si el primer comentario del documento aparece inmediatamente después de la declaración `<!DOCTYPE html>`, se tomará como la descripción del código.

El código HTML de entrada es el siguiente (el fichero está disponible en el apartado dedicado a esta tarea dentro del aula virtual):

```
1 <!DOCTYPE html>
2 <!-- Pagina principal de la asignatura de Computabilidad y Algoritmia --
3 >
4 <html>
5 <head>
6 <title>Computabilidad y Algoritmia</title>
7 </head>
8 <body>
9 <h1>Computabilidad y Algoritmia</h1>
10 <p>Bienvenidos a la asignatura.</p>
11 <a href="https://campusvirtual.u1l.es" class="enlace">Campus Virtual</a>
12 <h2>Practica 4</h2>
13 <p>En esta practica trabajaremos con expresiones regulares.</p>
14 
15 <ul>
16 <li>Expresiones regulares</li>
17 <li>Grupos de captura</li>
18 </ul>
19 <h2>Materiales</h2>
20 <p>Consulta los siguientes recursos.</p>
21 <a href="https://regex101.com" target="_blank">Regex101</a>
22 
23 <ol>
24 <li>Teoria de expresiones regulares</li>
25 <li>Ejercicios practicos</li>
26 <li>Documentacion de C++</li>
27 </ol>
28 <!--
29 Esta seccion contiene los recursos recomendados.
30 El contenido puede cambiar durante el curso.
31 -->
32 </body>
33 </html>
```

La salida del programa debería ser un fichero similar al siguiente:

```
1 PROGRAM: pagina.html
2
3 DESCRIPTION:
4 Pagina principal de la asignatura de Computabilidad y Algoritmia
5
6 STRUCTURE:
7 HTML: True
8 HEAD: True
9 BODY: True
10 DOCTYPE: HTML5
11
12 TAGS:
13 [Line 3] html
14 [Line 4] head
```

```
15 [Line 5] title
16 [Line 5] /title
17 [Line 6] /head
18 [Line 7] body
19 [Line 8] h1
20 [Line 8] /h1
21 [Line 9] p
22 [Line 9] /p
23 [Line 10] a
24 [Line 10] /a
25 [Line 12] p
26 [Line 12] /p
27 [Line 13] img
28 [Line 19] p
29 [Line 19] /p
30 [Line 20] a
31 [Line 20] /a
32 [Line 21] img
33 [Line 31] /body
34 [Line 32] /html
35
36 ATTRIBUTES:
37 [Line 10] a
38 href = "https://campusvirtual.ull.es"
39 class = "enlace"
40
41 [Line 13] img
42 src = "regex.png"
43 alt = "Expresiones regulares"
44 width = "400"
45
46 [Line 20] a
47 href = "https://regex101.com"
48 target = "_blank"
49
50 [Line 21] img
51 src = "logo.png"
52 alt = "Logo de la universidad"
53
54 COMMENTS:
55 [Line 2] DESCRIPTION
56 <!-- Pagina principal de la asignatura de Computabilidad y Algoritmia --
57 >
58 [Line 27-30]
59 <!--
60     Esta seccion contiene los recursos recomendados.
61     El contenido puede cambiar durante el curso.
62 -->
```

En caso de que no esté familiarizado con el lenguaje HTML, puede consultar una guía en este enlace [12]

4. Criterios de evaluación

Se señalan a continuación los aspectos más relevantes (la lista no es exhaustiva) que se tendrán en cuenta a la hora de evaluar esta práctica:

- Se valorará que el alumnado haya realizado, con anterioridad a la sesión de prácticas, y de forma efectiva, todas las tareas propuestas en este guión. Esto implicará que el programa compile y ejecute correctamente.
- También se valorará que, con anterioridad a la sesión de prácticas, el alumnado haya revisado los documentos que se enlazan desde este guión.
- Paradigma de programación orientada a objetos: se valorará que el alumnado haya identificado clases y objetos que permitan modelar adecuadamente el escenario de trabajo que se plantea.
- Paradigma de modularidad: se valorará que el programa se haya escrito de modo que las diferentes funcionalidades que se precisen hayan sido encapsuladas en métodos concretos cuya extensión textual se mantuviera acotada.
- Uso de expresiones regulares a través de la librería **regex**: se valorará el uso de expresiones regulares y de operaciones básicas como **match** y **search**.
- Se valorará que el código desarrollado siga el formato propuesto en esta asignatura para la escritura de programas en C++.
- Capacidad del programador(a) de introducir cambios en el programa desarrollado.

Si el alumnado tiene dudas respecto a cualquiera de estos aspectos, debiera acudir al foro de discusiones de la asignatura para plantearlas allí. Se espera que, a través de ese foro, el alumnado intercambie experiencias y conocimientos, ayudándose mutuamente a resolver dichas dudas. También el profesorado de la asignatura intervendrá en las discusiones que pudieran suscitarse, si fuera necesario.

Referencias

- [1] Transparencias del Tema 2 de la asignatura: Autómatas finitos y lenguajes regulares, <https://campusvirtual ull.es/2526/ingenieriaytecnologia/mod/resource/view.php?id=11856>
- [2] Tutorial de Regex: Aprender con ejemplos de expresiones regulares, <https://4geeks.com/es/lesson/regex-tutorial-regular-expression-ejemplo>
- [3] The Regex Library, <http://www.cplusplus.com/reference/regex>
- [4] RegexOne: Learn Regular Expressions with simple, interactive exercises, <https://regexone.com>
- [5] Regular Expressions in C++, <https://dzone.com/articles/introduction-to-regular-expression-with-modern-c>
- [6] Why Using the Greedy .* in Regular Expressions Is Almost Never What You Actually Want, <https://mariusschulz.com/blog/why-using-the-greedy-in-regular-expressions-is-almost-never-what-you-actually->
- [7] Expresiones regulares: cuantificadores perezosos, <https://learntutorials.net/es/cplusplus/topic/1681/expresiones-regulares>
- [8] Interactive Regular Expressions, <https://regex101.com>
- [9] Makefile Tutorial: <http://www.cs.colby.edu/maxwell/courses/tutorials/maketutor>
- [10] C++ Makefile Tutorial: <https://makefiletutorial.com>
- [11] Google C++ Style Guide, <https://google.github.io/styleguide/cppguide.html>
- [12] Tutorial de HTML https://developer.mozilla.org/es/docs/Learn_web_development/Core/Structuring_content/Basic_HTML_syntax